

SECURELINK

Plateforme de detection de phishing
pour la protection et l'education du grand public

RAPPORT TECHNIQUE ET FONCTIONNEL

Architecture, pipeline d'analyse, benchmark et preparation aux questions du jury

PARTIE I — APPROCHE TECHNIQUE

Architecture, pipeline d'analyse, scoring hybride, securite et validation

1. Architecture globale du systeme

SecureLink est construit sur une architecture trois-tiers classique augmentee de services externes optionnels. Le frontend React communique avec le backend FastAPI via une API REST prefixee /api. Le backend orchestre les 12 couches d'analyse, calcule le score hybride, et persiste les resultats dans MongoDB. Trois services externes enrichissent l'analyse sans jamais etre indispensables.

1.1 Les quatre briques et leurs roles

Brique	Technologie	Role principal	Chiffres clés
Frontend	React 19, Tailwind CSS, shadcn/ui	Interface utilisateur, routing SPA, gestion d'etat	11 pages, 3 contextes (Auth, Langue, Theme)
Backend	FastAPI (Python), uvicorn	API REST, orchestration des couches, scoring	38 endpoints, validation Pydantic, doc OpenAPI auto
Base de donnees	MongoDB (Motor async)	Stockage documentaire, schema flexible	11 collections, pas de migrations
Services externes	Safe Browsing, Gemini 1.5 Flash, Pappers	Detection, chatbot IA, verification entreprise FR	3 APIs, toutes optionnelles

1.2 Justification des choix technologiques

Technologie	Alternative ecartee	Raison du rejet
FastAPI	Django REST Framework	Django trop lourd pour une approche API-first ; pas d'async natif sans extensions
FastAPI	Express.js (Node)	Manque de typage natif ; Python necessaire pour scikit-learn
React 19	Vue.js / Svelte	Ecosysteme de composants plus riche (shadcn/ui, Recharts) ; communaute plus large
MongoDB	PostgreSQL	Les resultats d'analyse sont des documents JSON imbriques de structure variable, mal adaptes a un schema relationnel fixe
scikit-learn (RandomForest)	XGBoost	Gain marginal sur des features tabulaires ; RandomForest offre feature_importances_ pour l'interpretabilite
scikit-learn (RandomForest)	Deep Learning	Overkill sans GPU ; boite noire incompatible avec l'objectif pedagogique
Playwright	Puppeteer	Puppeteer est restreint a Node.js ; le backend est en Python
Playwright	Selenium	API vieillissante, plus lent, moins fiable en headless

Technologie	Alternative ecartee	Raison du rejet
Gemini 1.5 Flash	GPT-4 / Claude	Ratio qualite/cout optimal ; gratuit avec quotas genereux pour un chatbot educatif

1.3 Deux vigilances architecturales

Vigilance 1 — Dégradation gracieuse. Chaque service externe est optionnel. Si Google Safe Browsing est indisponible, l'analyse continue avec les couches restantes et le score est recalculé sur les poids disponibles. Si la cle Gemini n'est pas configurée, le chatbot renvoie un message d'erreur explicite mais le reste de l'application fonctionne. Si Playwright n'est pas installé, le screenshot est marqué "non disponible" sans erreur 500. Ce principe garantit qu'aucune panne d'API externe ne provoque un crash du système.

Vigilance 2 — Asynchronisme et non-blocage. Les vérifications réseau (DNS, SSL, WHOIS, rDNS) sont exécutées en parallèle via asyncio avec un timeout global de 10 secondes. Si une vérification individuelle (typiquement WHOIS) dépasse ce délai, elle est simplement marquée comme timeout sans pénalité sur le score. Le screenshot est exécuté dans un subprocess Python isolé (screenshot_worker.py) pour éviter les conflits de boucle événementielle entre Playwright et uvicorn, notamment sous Windows.

2. Le pipeline d'analyse : les 12 couches

Lorsqu'un utilisateur soumet une URL, l'orchestrateur `url_analyzer.py` démarre simultanément 12 couches d'analyse indépendantes. Chaque couche produit un signal spécifique. Les signaux sont ensuite combinés par la formule de scoring hybride pour produire un score final de 0 à 100, traduit en feu tricolore.

2.1 Vue complete des 12 couches

#	Couche	Module Python	Methode	Signal produit
1	Modele ML	<code>ml_model.py</code>	RandomForest (100 arbres, 20 features)	Probabilite phishing [0-100]
2	Regles deterministes	<code>url_analyzer.py</code>	15 heuristiques (IP, TLD, entropie...)	Score de regles + flags binaires
3	DNS	<code>network_checks.py</code>	<code>socket.getaddrinfo()</code>	Le domaine existe-t-il ?
4	SSL/TLS	<code>network_checks.py</code>	<code>ssl.create_default_context()</code>	Certificat valide, emetteur, dates
5	Chaine de certificats	<code>network_checks.py</code>	Analyse du leaf certificate	Auto-signé ? DV/OV/EV ? SANs ?
6	WHOIS	<code>network_checks.py</code>	Bibliotheque whois	Age du domaine, registrar
7	Contenu HTML	<code>content_analyzer.py</code>	BeautifulSoup (HTTP GET + parsing)	Formulaires login, scripts suspects, usurpation de marque
8	Google Safe Browsing	<code>safe_browsing.py</code>	API v4 POST /v4/threatMatches:find	Menace connue oui/non
9	Typosquatting	<code>typosquatting_detector.py</code>	Levenshtein + substitution + homoglyphes	Imitation d'un domaine connu ?
10	Redirections	<code>content_analyzer.py</code>	<code>requests.get(allow_redirects=True)</code>	Cross-domain, multi-hop, downgrade HTTPS
11	Kits de phishing	<code>phishing_kit_detector.py</code>	Fingerprinting regex (5 kits, 8+ patterns)	Signature de kit connu detecte ?
12	Screenshot	<code>page_screenshot.py</code> + <code>screenshot_worker.py</code>	Playwright (Chromium headless, subprocess)	Capture JPEG de la page

2.2 Detail : Couche 1 — Modele ML (RandomForest)

Le modele extrait 20 features directement de la structure de l'URL, sans acces reseau. La prediction prend moins d'une milliseconde. Les hyperparametres sont : `n_estimators=100`, `max_depth=15`, `min_samples_split=5`, `random_state=42`. Le dataset d'entrainement comprend 3 000 URLs synthetiques (50% legitimes, 50% phishing), avec un split 80/20. L'accuracy sur le jeu de test est de 98,8%.

#	Feature	Justification
0	url_length	URLs de phishing souvent plus longues (obfuscation)
1	domain_length	Domaines legitimes = courts et memorisables
2	has_https	HTTPS = signal faible de legitime
3	num_dots	Beaucoup de sous-domaines = suspect
4	num_hyphens	Tirets frequents dans les domaines de phishing
5	has_at	Le caractere @ redirige vers un autre domaine
6	digits_in_domain	Domaines legitimes contiennent rarement des chiffres
7	has_ip	IP au lieu d'un nom de domaine = tres suspect
8	suspicious_tld	TLDs .tk, .xyz, .ml surrepresentes en phishing
9	domain_entropy	Entropie de Shannon elevee = domaine genere aleatoirement
10-14	path_length, query_length, has_port, subdomain_count, special_chars	Structure de l'URL : profondeur, parametres, ports non-standard
15-19	has_redirect, shortened_url, num_slashes, digits_in_path, suspicious_keywords	Signaux comportementaux : redirection cachee, raccourcisseur, mots-cles (login, verify)

2.3 Detail : Couche 2 — Regles deterministes

Les regles statiques capturent des certitudes que le ML pourrait sous-ponderer. Par exemple, une adresse IP dans le domaine est toujours suspecte, sans exception.

Regle	Points	Logique
IP dans le domaine	+25	Aucun site grand public legitime n'utilise une IP
TLD suspect (.tk, .xyz, .ml)	+15	Domaines gratuits massivement exploites en phishing
URL tres longue (>75 car.)	+10	Technique d'obfuscation frequente
3+ sous-domaines	+15	Imitation (login.google.com.evil.tk)
Entropie de Shannon > 4.0	+15	Domaine genere aleatoirement
Mots-cles suspects (login, verify, secure)	+10	Appat psychologique classique
Pattern de redirection (//) dans le path	+15	Redirection cachee
URL raccourcie (bit.ly, tinyurl)	+10	Masque la vraie destination
HTTPS present (bonus)	-10	Signal positif de legitime
URL courte et propre (<40 car., bonus)	-5	Profil rassurant

2.4 Detail : Couches 3-6 — Verifications reseau

Verification	Methode	Signal positif (bonus)	Signal negatif (penalite)
DNS	socket.getaddrinfo()	Domaine resolu = existe	Non resolu = potentiellement hors ligne
SSL	ssl.create_default_context()	Certificat valide, TLS 1.2+	+20 si pas de SSL
Chaine de certificats	Analyse leaf certificate	OV/EV = identite verifiee	+15 si auto-signé ou SANs absents
WHOIS	Bibliotheque whois	Domaine > 10 ans : - 15 pts	+15 si < 30 jours
rDNS	socket.gethostbyaddr()	Reverse DNS coherent	+10 si incoherent

Le scoring reseau va de -30 (tres sur) a +40 (tres suspect). Les verifications sont parallelisees avec `asyncio.as_completed()` et un timeout global de 10 secondes.

2.5 Detail : Couche 7 — Analyse du contenu HTML

- Detection de formulaires de login : presence de champs `input type="password"`.
- Usurpation de marque : logo ou nom de marque present dans le HTML mais incoherent avec le domaine.
- Scripts suspects : obfuscation, `eval()`, `document.cookie`, exfiltration.
- Comptage de domaines externes : si la page charge des ressources depuis de nombreux domaines differents.
- Extraction de signaux d'entreprise : SIREN, raison sociale, pour alimenter la verification Pappers (couche 5 fonctionnelle).

2.6 Detail : Couche 9 — Detection de typosquatting

L'algorithme compare le domaine soumis a une liste de 50+ marques connues via trois mecanismes complementaires. La distance de Levenshtein mesure le nombre minimum d'operations (insertion, suppression, substitution) pour transformer un mot en un autre. Les substitutions courantes sont verifiees explicitement : o vers 0, l vers 1, rn vers m, vv vers w. Les homographes Unicode (L majuscule ressemblant a l majuscule) sont detectes par une table de correspondance dediee. Si le score de similarite depasse le seuil, le typosquatting est signale avec identification de la marque ciblee et lien vers le vrai site.

2.7 Detail : Couche 11 — Fingerprinting de kits de phishing

Kit	Cible	Technique de detection
16Shop	Apple, Amazon, PayPal	Anti-bot + redirection, signatures HTML specifiques
LogoKit	Dynamique (multi-marques)	Generation automatique de logos, patterns URL reconnaissables
Chase Kit	Banque Chase	Structure de faux portail bancaire, formulaire multi-etapes
Office365 Kit	Microsoft	Faux portail SSO, capture de tokens OAuth
Telegram Kit	Variable	Exfiltration via bot Telegram, patterns de callback

En complement des 5 kits nommes, 8 patterns structurels sont detectes : logo en base64, exfiltration AJAX, redirection obfusquee, formulaire PHP mailer, formulaire multi-etapes, fausse animation de chargement, compte a rebours d'urgence, data URI form.

3. Le systeme de scoring hybride

3.1 La formule

$$\text{Score} = (\text{ML} \times 0,35) + (\text{Regles} \times 0,20) + (\text{Reseau} \times 0,15) + (\text{Contenu} \times 0,15) + (\text{SafeBrowsing} \times 0,10) + (\text{Kit} \times 0,05) + \text{penalites} + \text{boost de legitime}$$

Couche	Poids	Justification
ML	35%	Seule couche évaluant les 20 features simultanément avec des non-linearités
Regles	20%	Signaux binaires forts (IP, TLD), corrige les angles morts du ML
Reseau	15%	Fiable mais lent ; certains domaines légitimes peuvent échouer temporairement
Contenu	15%	Puissant mais pas toujours disponible (page qui ne répond pas, JS-only)
Safe Browsing	10%	Tres fiable quand Google a l'information, mais base non exhaustive pour les sites nouveaux
Kit Detection	5%	Rare mais décisif ; le hard override compense le faible poids

3.2 Hard overrides

Certains signaux sont si forts qu'ils imposent un plancher de danger indépendamment du score calculé :

- Google Safe Browsing signale une menace : score minimum force à 75.
- Kit de phishing connu détecte : score minimum force à 70.
- Typosquatting détecte : score minimum force à 70.

3.3 Boost de legitime

Pour éviter les faux positifs sur les sites légitimes à structure complexe (URLs longues, sous-domaines multiples), un système de bonus s'active lorsque 3 signaux de confiance ou plus convergent simultanément :

Signal de confiance	Bonus accorde
Age du domaine > 2 ans (WHOIS)	-15 points
Age du domaine > 1 an	-5 points
SSL valide (TLS 1.2/1.3)	-5 points
Google Safe Browsing clean	-5 points
Domaine dans la whitelist (60+ domaines)	-20 points

Condition d'activation : le bonus ne s'applique que si au moins 3 signaux sont réunis. Un seul signal positif (par exemple SSL valide) ne suffit pas à annuler un véritable phishing. Exemple concret : wikipedia.org cumule 5-6 signaux, produisant un bonus de -45 à -50 points.

3.4 Seuils du feu tricolore

Score	Couleur	Signification
0 - 29	Vert (Sur)	Aucune couche n'a detecte de signal significatif
30 - 54	Orange (Prudence)	Au moins une couche a detecte quelque chose, pas de certitude
55 - 100	Rouge (Danger)	Signal fort d'au moins une couche OU accumulation de signaux faibles
N/A	Gris (Hors ligne)	DNS non resolu et aucun signal de phishing intentionnel

Les seuils ont ete calibres par grid search sur 7 valeurs (20, 25, 30, 35, 40, 45, 50). Les seuils 20 a 35 donnent tous 100% de F1-Score. Le seuil 30 est retenu pour le meilleur compromis detection/tolerance.

3.5 Detection hors ligne

Condition	Verdict
DNS echoue + pas de typosquatting + pas de TLD suspect + pas de mots-cles phishing	HORS LIGNE (gris) — score remis a 0
DNS echoue + typosquatting detecte	DANGEREUX (rouge) — phishing intentionnel
DNS echoue + TLD suspect ou mots-cles phishing	Score maintenu — signaux de phishing presents

4. Securite de l'application

Menace	Protection	Implementation
SSRF	Verification IP resolue avant chaque requete sortante	ipaddress.ip_address().is_private bloque loopback, prive, reserve, link-local
Brute-force login	Rate limiting	10 tentatives/minute via SlowAPI
Abus analyseur	Rate limiting	20 analyses/heure par IP
Abus chatbot	Rate limiting	30 messages/heure par IP
CORS	Origines strictes	Variable d'environnement, pas de wildcard en production
Injection	Validation Pydantic	Tous les inputs valides par BaseModel avec contraintes de type et longueur
Vol de session	JWT stateless	Signe avec secret, duree 24h, bcrypt cost 12
Mot de passe faible	Politique de mots de passe	Minimum 8 caracteres, 1 majuscule, 1 minuscule, 1 chiffre

5. Validation empirique : le benchmark

5.1 Composition du dataset de test

Le script benchmark.py teste automatiquement 74 URLs (45 legitimes + 29 phishing) couvrant des categories variees :

Categorie	Exemples	Nombre
Tech majeur	google.com, youtube.com, github.com	12
E-commerce	amazon.com, amazon.fr, ebay.com	5
Finance	paypal.com, stripe.com, boursorama.com	4
Gouvernement FR	impots.gouv.fr, service-public.fr, ameli.fr	3
Outils SaaS	slack.com, notion.so, figma.com	7
Telecom FR	orange.fr, free.fr, sfr.fr	3
URLs avec parametres	google.com/search?q=..., youtube.com/watch?v=...	5
Typosquatting	g00gle.com, faceb00k-verify.com	8
IP dans URL	192.168.1.1/login.php	3
TLD suspects	xk3jf9s.xyz, secure-banking.tk	5
Domaine aleatoire	zx9f3k2m.com, a1b2c3d4e5.net	3

5.2 Resultats

Matrice de confusion : 45 Vrais Negatifs | 29 Vrais Positifs | 0 Faux Positifs | 0 Faux Negatifs.
Accuracy : 100%. Precision : 100%. Rappel : 100%. F1-Score : 100%.

5.3 Calibration par grid search

7 seuils testes (20, 25, 30, 35, 40, 45, 50). Les seuils 20 a 35 produisent tous un F1-Score de 100%. Le seuil 30 est retenu comme meilleur compromis. Les poids de la formule de scoring (35/20/15/15/10/5) n'ont pas necessite d'ajustement apres calibration.

PARTIE II — APPROCHE FONCTIONNELLE

Fonctionnalités, parcours utilisateur, chatbot, extensions, gamification

6. Les 16 fonctionnalités implementees

ID	Fonctionnalite	Description courte
F1	Analyse de lien	Coller une URL, analyse automatique en < 3 secondes
F2	Scoring hybride	Score 0-100 avec feu tricolore (vert/orange/rouge/gris)
F3	Explication pedagogique	Resume en langage simple de chaque risque detecte
F4	Detection de typosquatting	Identification du domaine imite + lien vers le vrai site
F5	Authentification JWT	Inscription/connexion email, mot de passe hache bcrypt
F6	Dashboard utilisateur	Historique, statistiques personnelles, progression XP
F7	Dashboard administrateur	Vue d'ensemble, gestion utilisateurs, moderation
F8	Chatbot IA (SecureBot)	Assistant conversationnel pour analyser des messages suspects
F9	E-learning	Modules interactifs sur les types de phishing et la prevention
F10	Simulation de phishing	Exercices pratiques pour entrainer l'utilisateur a detecter le phishing
F11	Gamification	XP (+10 par analyse, +25 si menace), badges, 5 niveaux, leaderboard
F12	Extension Chrome	Manifest V3, analyse de l'onglet actif ou clic droit sur un lien
F13	Extension Firefox	Manifest V2, meme logique avec API browser.*
F14	Capture d'ecran	Screenshot JPEG via Playwright headless (subprocess isole)
F15	Internationalisation	Interface en francais et en anglais (LanguageContext)
F16	Mode clair/sombre	Theme adaptatif via ThemeContext

7. Parcours utilisateur

7.1 Premiere visite (sans compte)

L'utilisateur arrive sur la HomePage et voit immediatement la barre d'analyse avec le message "Collez ici le lien suspect que vous souhaitez verifier". Il peut analyser sans inscription. Le resultat s'affiche avec le verdict en feu tricolore, 3 a 5 points cles en langage courant, et le screenshot de la page. Le rapport technique complet est accessible via un bouton optionnel.

7.2 Premiere connexion (avec compte)

Apres inscription, un tutoriel en 4 ecrans (OnboardingModal) presente l'application. L'utilisateur accede ensuite a son dashboard personnel avec ses statistiques, son niveau, ses badges et son historique. Chaque analyse rapporte 10 XP, et 25 XP si une menace est detectee.

8. SecureBot — Le chatbot IA

SecureBot utilise Google Gemini 1.5 Flash avec un system prompt specialise en cybersécurité. L'historique de session (10 derniers messages) est injecte dans le contexte du LLM. Le chatbot detecte automatiquement si l'utilisateur a colle un message suspect (plus de 100 caracteres, presence d'URLs, structure de copier-coller) et produit une analyse structuree : verdict, techniques detectees, signaux d'alerte, conseils d'action, et conseil pedagogique.

Rate-limite a 30 messages/heure par IP. Cantonne a la cybersécurité (redirige poliment si hors sujet). Sessions identifiees par UUID, 200 sessions max en memoire avec eviction LRU.

9. Extensions navigateur

Aspect	Chrome/Edge (Manifest V3)	Firefox (Manifest V2)
Manifest	manifest_version: 3	manifest_version: 2
Popup	action.default_popup	browser_action.default_popup
Background	service_worker	scripts array
API	chrome.* (callbacks)	browser.* (Promises)
Analyse	POST /api/analyze (meme endpoint que le web)	POST /api/analyze (meme endpoint)
Resultat	Badge colore + 4 points cles max + score	Badge colore + 4 points cles max + score

Les extensions offrent deux modes : analyse de l'onglet actif (URL pre-remplie) et clic droit sur un lien (menu contextuel). La fonction buildSummary() extrait intelligemment les informations les plus pertinentes de la reponse API.

10. Base de donnees — Schema MongoDB

Collection	Contenu principal	Relation
users	Email, password hash (bcrypt), XP, niveau, badges, is_admin	Cle primaire
analyses	URL, scores, feu tricolore, details de chaque couche	user_id (nullable si anonyme)
feedback	Analyse ID, avis utilisateur (phishing oui/non)	Liee a analyses et users
chat_messages	Session ID, role (user/assistant), contenu, analyse optionnelle	Groupees par session_id
quiz_progress	Module ID, score, XP gagne	Liee a users
simulation_results	Scenario ID, reponse, XP gagne	Liee a users
contributions	URL signalee, statut (pending/validated/rejected)	Liee a users
training_data	URLs pour re-entrainement du modele ML	Alimentee par feedback/contributions

Conformite donnees : aucun contenu HTML analyse n'est stocke. Seuls les scores et metadonnees sont conserves. Les analyses sont anonymisees pour les utilisateurs non connectes.

PARTIE III — GLOSSAIRE TECHNIQUE

Termes liés à l'analyse, au scoring et au benchmark

Termes fondamentaux

Terme	Definition	Contexte dans SecureLink
RandomForest	Algorithme de Machine Learning supervise qui aggrege les decisions de multiples arbres de decision. Chaque arbre vote, la majorite l'emporte.	Couche 1 du moteur d'analyse. 100 arbres, 20 features, prediction < 1ms.
Feature (caracteristique)	Variable d'entree utilisee par le modele ML pour faire sa prediction. Chaque URL est transformee en un vecteur de 20 features numeriques.	url_length, domain_entropy, has_ip, suspicious_tld, etc.
Feature importance	Mesure de la contribution de chaque feature a la decision du modele. Le RandomForest calcule cela nativement via feature_importances_.	Utilisee pour expliquer a l'utilisateur pourquoi un lien est suspect (dimension pedagogique).
Entropie de Shannon	Mesure mathematique du desordre ou de l'aleatoire dans une chaine de caracteres, exprimee en bits par caractere. Plus l'entropie est elevee, plus la chaine est aleatoire.	Un domaine genere aleatoirement (xk3jf9s.xyz) a une entropie elevee (> 4.0), ce qui est un signal de phishing.
Distance de Levenshtein	Nombre minimum d'operations (insertion, suppression, substitution) pour transformer un mot en un autre.	Utilisee pour detecter le typosquatting : si la distance entre un domaine et une marque connue est faible, c'est suspect.
Homoglyphe	Caractere visuellement identique ou tres proche d'un autre mais avec un code Unicode different. Exemple : I (majuscule) et l (minuscule).	Table de substitution dediee ajoutee au detecteur de typosquatting pour capturer paypal vs paypal.
Typosquatting	Enregistrement d'un nom de domaine visuellement proche d'une marque connue pour tromper les utilisateurs. Exemple : g00gle.com, amaz0n.com.	Couche 9 du moteur. Detection par Levenshtein + substitutions + homoglyphes. Hard override a 70 si detecte.

Termes de scoring et calibration

Terme	Definition	Contexte dans SecureLink
Scoring hybride	Methode de calcul du score final qui combine plusieurs sources de signaux avec des poids differents, au lieu de se fier a une seule methode.	Formule : ML(35%) + Regles(20%) + Reseau(15%) + Contenu(15%) + SB(10%) + Kit(5%).
Hard override	Regle qui impose un score minimum quand un signal tres fort	Si Google dit menace : min 75. Si kit detecte : min 70. Si typosquatting : min 70.

Terme	Definition	Contexte dans SecureLink
	est detecte, independamment du score calcule par la formule.	
Boost de legitime	Mecanisme qui reduit le score quand plusieurs signaux de confiance convergent, pour eviter les faux positifs sur les sites legitimes.	Active si 3+ signaux reunis (age domaine, SSL, Safe Browsing clean, whitelist).
Grid search	Methode d'optimisation qui teste systematiquement toutes les combinaisons de parametres dans une grille predefinee pour trouver la meilleure.	7 valeurs de seuils testees (20 a 50). Le seuil 30 retenu pour le meilleur compromis.
Degradation gracieuse	Capacite d'un systeme a continuer de fonctionner correctement meme si certains composants sont indisponibles.	Chaque API externe est optionnelle. Le score est recalculé avec les couches restantes.
Feu tricolore	Traduction du score numerique en verdict visuel comprehensible par un non-expert.	Vert (0-29), Orange (30-54), Rouge (55-100), Gris (hors ligne).

Termes de benchmark et validation

Terme	Definition	Contexte dans SecureLink
Benchmark	Test systematique d'un systeme sur un dataset dont les reponses correctes sont connues a l'avance, pour mesurer sa performance.	74 URLs (45 legitimes + 29 phishing) testees automatiquement par benchmark.py.
Accuracy	Proportion de predictions correctes parmi l'ensemble des predictions. Formule : $(VP + VN) / (VP + VN + FP + FN)$.	100% sur le benchmark de 74 URLs.
Precision	Parmi les URLs classees phishing, proportion de celles qui sont reellement du phishing. Formule : $VP / (VP + FP)$.	100%. Signifie : quand SecureLink dit danger, c'est toujours vrai.
Rappel (Recall)	Parmi les URLs reellement phishing, proportion de celles que le systeme a detectees. Formule : $VP / (VP + FN)$.	100%. Signifie : SecureLink ne laisse passer aucun phishing.
F1-Score	Moyenne harmonique de la precision et du rappel. Formule : $2 \times (Precision \times Rappel) / (Precision + Rappel)$.	100%. Combine les deux metriques en un score unique.
Matrice de confusion	Tableau 2x2 qui croise les predictions du systeme avec la realite. 4 cases : Vrais Positifs, Vrais Negatifs, Faux Positifs, Faux Negatifs.	45 VN, 29 VP, 0 FP, 0 FN.
Faux positif (FP)	Site legitime que le systeme classe a tort comme phishing. C'est le	0 sur le benchmark. Resolu par le boost de legitime.

Terme	Definition	Contexte dans SecureLink
	probleme le plus penalisant : il fait perdre confiance a l'utilisateur.	
Faux negatif (FN)	Site de phishing que le systeme ne detecte pas. C'est le probleme le plus dangereux : l'utilisateur clique sur un lien malveillant en confiance.	0 sur le benchmark. Resolu par les hard overrides et l'approche multi-couches.
Dataset synthetique	Donnees generees artificiellement selon des regles, par opposition a des donnees collectees en conditions reelles.	Le dataset d'entrainement ML (3000 URLs) est synthetique. Le benchmark (74 URLs) utilise des URLs reelles.
Overfitting (surapprentissage)	Le modele apprend trop bien les donnees d'entrainement et generalise mal sur de nouvelles donnees.	Evite par le RandomForest (agregation de 100 arbres) et le split 80/20.

PARTIE IV — QUESTIONS PROBABLES DE L'EXAMINATEUR

Reponses preparees, classees par theme

Bloc 1 : Questions sur le modele ML

"Pourquoi RandomForest et pas un reseau de neurones ?"

Pour trois raisons. Premierement, l'interpretabilite : le RandomForest fournit nativement les `feature_importances_`, ce qui nous permet d'expliquer a l'utilisateur quelles caracteristiques ont pese dans la decision. C'est essentiel pour la dimension pedagogique du projet. Deuxiemement, pas de GPU requis : le deploiement est simple sur n'importe quel serveur. Troisiemement, la performance est suffisante : 98,8% d'accuracy sur des features tabulaires. Un reseau de neurones n'apporterait pas de gain significatif sur ce type de donnees, et il transformerait le systeme en boite noire.

"Le dataset d'entrainement est synthetique. Ca ne pose pas un probleme de generalisation ?"

C'est une limite identifiee et documentee. C'est precisement pour ca que le ML ne pese que 35% du score final. Les 11 autres couches, dont Google Safe Browsing (base de donnees mondiale mise a jour en temps reel), les verifications reseau reelles (DNS, SSL, WHOIS), et le fingerprinting de kits de phishing, compensent ce biais. Le benchmark de validation sur 74 URLs reelles confirme que le systeme complet fonctionne correctement malgre cette limite.

"3 000 URLs d'entrainement, c'est peu pour du ML. Comment evitez-vous le surapprentissage ?"

Le RandomForest est resistant au surapprentissage grace a l'agregation de 100 arbres de decision independants (bagging). Le split 80/20 permet de mesurer la generalisation sur des donnees non vues. Et surtout, le ML est complete par 11 autres couches : meme si le modele ML se trompe sur un cas particulier, le scoring hybride ponderecompense par les autres signaux.

"Pourquoi 20 features et pas plus ?"

Les 20 features couvrent les signaux structurels les plus discriminants de l'URL : longueur, entropie, sous-domaines, TLD, caracteres speciaux, mots-cles suspects. Ajouter des features redondantes (par exemple le nombre de voyelles) n'ameliorerait pas la precision et risquerait d'introduire du bruit. Le `feature_importances_` du modele confirme que les 20 features sont toutes utilisees par les arbres de decision.

Bloc 2 : Questions sur le benchmark

"74 URLs, c'est un benchmark limite. Comment peut-on generaliser ?"

C'est une limite assumee et documentee. Nos 74 URLs couvrent des categories variees et representatives : sites tech majeurs, e-commerce, finance, gouvernement francais, typosquatting, IP dans URL, TLD suspects, domaines aleatoires. Le 100% d'accuracy est le resultat du scoring hybride multi-couches, pas du ML seul. La perspective a court terme est d'etendre a 500+ URLs avec PhishTank et OpenPhish.

"100% d'accuracy, c'est suspect. Vous n'avez pas overfit votre seuil sur le benchmark ?"

Les seuils ont ete calibres par grid search sur 7 valeurs (20 a 50). Les seuils 20 a 35 donnent tous 100% de F1-Score. Le seuil 30 n'est donc pas un point isole : il appartient a une plage de 4 valeurs qui produisent le meme resultat. Le 100% est le fruit du scoring hybride, du boost de legitimité (qui elimine les faux positifs) et des hard overrides (qui eliminent les faux negatifs). Ce ne serait pas le cas avec le ML seul.

"Le benchmark est-il reproductible ?"

Oui. Le script benchmark.py peut être relancé à tout moment. Les résultats peuvent varier légèrement selon l'état des serveurs WHOIS et la disponibilité des APIs externes, mais le benchmark gère ces variations grâce à la dégradation gracieuse.

Bloc 3 : Questions sur l'architecture

"Pourquoi MongoDB et pas PostgreSQL ?"

Les résultats d'analyse sont des documents JSON imbriqués dont la structure varie selon les couches actives. Si Safe Browsing est indisponible, le champ `safe_browsing` est null. Si le typosquatting est détecté, un objet supplémentaire apparaît. Ce schéma variable est naturellement adapté au modèle documentaire de MongoDB. Un schéma relationnel fixe nécessiterait de nombreuses colonnes nullable et des migrations à chaque évolution du pipeline.

"Comment gerez-vous la concurrence sur les analyses parallèles ?"

Les 12 couches sont exécutées en parallèle via `asyncio`. Les couches locales (ML, règles, typosquatting) sont quasi-instantanées (<1ms). Les couches réseau (DNS, SSL, WHOIS, rDNS) utilisent `asyncio.as_completed()` avec un timeout de 10 secondes. Si une couche dépasse le timeout, elle est marquée comme indisponible sans bloquer les autres. Le scoring est recalculé sur les couches qui ont répondu.

"Pourquoi avoir isolé Playwright dans un subprocess ?"

Sur Python 3.14 sous Windows, l'API `async` de Playwright entre en conflit avec la boucle événementielle d'uvicorn (`NotImplementedError` sur `asyncio.create_subprocess_exec`). Le `screenshot_worker.py` utilise l'API synchrone de Playwright dans un processus Python complètement isolé, communiquant par `subprocess.run()` et JSON sur `stdout`. Avantages : isolation totale des event loops, stabilité (si Playwright crash, le serveur continue), timeout de 25 secondes, compatibilité Windows/macOS/Linux.

Bloc 4 : Questions sur la sécurité et le RGPD

"Comment protégez-vous contre les attaques SSRF ?"

Avant chaque requête HTTP sortante (contenu, screenshot), le système résout l'adresse IP du domaine et vérifie via `ipaddress.ip_address()` qu'elle n'est pas privée, loopback, réservée ou link-local. Un attaquant ne peut pas soumettre `http://127.0.0.1:8080/admin` pour scanner le réseau interne.

"RGPD, concrètement, vous faites quoi ?"

Aucun contenu HTML analysé n'est stocké en base. Seuls les scores numériques et les métadonnées (URL, timestamp, feu tricolore) sont conservés. Les analyses réalisées sans compte sont anonymisées. L'architecture est stateless (JWT, pas de stockage serveur de sessions).

"Les rate limits sont-elles suffisantes pour une mise en production ?"

En contexte PFE, les limites actuelles (20 analyses/h, 30 chats/h, 10 logins/min) sont adaptées à un usage démontable. En production, il faudrait ajouter une identification par compte (pas seulement par IP) et des quotas différenciés par offre (Gratuit/Pro/Entreprise).

Bloc 5 : Questions sur le produit et le marché

"Votre modèle freemium est-il viable ?"

Le modèle repose sur l'acculturation gratuite (grand public) pour créer un flux d'utilisateurs, et la monétisation B2B (PME, grands comptes) pour générer des revenus. Le coût marginal par analyse est quasi-nul (pas de GPU, pas d'API payante en mode gratuit). Le marché français seul représente

50M+ internautes et 4M+ PME. Des solutions comparables en cybersécurité B2B (KnowBe4, Proofpoint) valident ce positionnement.

"Qu'est-ce qui empêche un concurrent de copier votre approche ?"

La barrière n'est pas technologique, elle est dans la combinaison. N'importe qui peut faire du ML sur des URLs ou intégrer Safe Browsing. La valeur de SecureLink est dans l'orchestration des 12 couches, la calibration empirique du scoring, la pédagogie intégrée, et la gamification. C'est l'assemblage qui est difficile à reproduire, pas les composants individuels.

"Pourquoi un utilisateur choisirait SecureLink plutôt que VirusTotal ?"

Parce qu'il n'est pas expert en sécurité. VirusTotal affiche des dizaines de résultats de moteurs antivirus incompréhensibles. SecureLink montre un feu tricolore, 3 points clés en langage courant, et un screenshot. C'est la même différence qu'entre un rapport de radiologie et l'explication du médecin.

Bloc 6 : Questions sur les limites et perspectives

"Quelles sont vos trois plus grandes limites ?"

Premièrement, le dataset d'entraînement ML est synthétique. Deuxièmement, le benchmark porte sur 74 URLs, ce qui n'est pas exhaustif. Troisièmement, l'analyse est réactive (à la demande), pas proactive. Ces trois limites sont identifiées, documentées et des perspectives de résolution existent : enrichissement du dataset avec PhishTank, extension du benchmark à 500+ URLs, et monitoring continu via l'extension navigateur.

"Si vous aviez 6 mois de plus, que feriez-vous en priorité ?"

Trois choses. Publier les extensions sur les stores Chrome et Firefox pour toucher les vrais utilisateurs. Étendre le benchmark à 500+ URLs réelles pour renforcer la preuve de fiabilité. Et mettre en place un mécanisme de calibration automatique des poids du scoring à partir du feedback utilisateur (le bouton 'Ce résultat est-il correct ?' existe déjà dans l'interface).

"L'analyse ne couvre pas le JavaScript. C'est un problème ?"

Oui, les pages SPA (Single Page Application) qui rendent leur contenu entièrement en JavaScript ne sont pas entièrement analysées côté contenu HTML. C'est une limite connue. Cependant, le screenshot via Playwright montre le rendu final tel que l'utilisateur le verrait. Et les 11 autres couches (ML, règles, DNS, SSL, WHOIS, Safe Browsing, typosquatting, kits, redirections) analysent l'URL indépendamment du contenu de la page.